Arquitectura de la FFT del bloque de CD — Recomendación y especificación

Oct 31, 2026 · @Wanda Molina

Resumen

Recomiendo una **FFT radix-2² propia**, no la IP de Xilinx, montada en la ZCU208 con **8 carriles a 368,64 MHz** (opción B, unos 228 DSP) o, si el presupuesto de DSP lo permite, con **16 carriles a 184,32 MHz** (opción A, unos 426 DSP). La cadena es FFT DIF → producto por $\bar{H}$ en orden bit invertido → IFFT DIT, sin buffers de reordenamiento, y da los **mismos bits** con cualquier cantidad de carriles.

Después de una introducción que explica radix-2² en hardware desde cero, el documento tiene dos partes que se apoyan una en la otra:

1. **La especificación general** (Arquitectura, Descomposición por etapas y Aritmética bit a bit) define el grafo, los redondeos y el escalado. No depende de la placa ni de P, y es lo que tiene que replicar el modelo de punto fijo.
2. **La implementación en la ZCU208** (desde Consideraciones de la ZCU208 hasta Flujo continuo) decide cuántos carriles y qué reloj usa esa misma especificación para sostener 1,47456 GSa/s, y cuánto cuesta.

Como la segunda parte solo cambia en qué ciclo y en qué carril se calcula cada nodo del grafo, elegir A o B no cambia el modelo bit a bit.

- **Cambio de N en ejecución:** el hardware se dimensiona para L_max = 4096; las L más chicas saltean etapas del principio de la FFT y del final de la IFFT.
- **Anchos:** entrada y salida de 14 bits (los convertidores), datos internos de 20 y twiddles y $\bar{H}$ de 18. Con 20 bits la FFT agrega menos de la mitad del error de cuantizar la entrada, sin costo de DSP.
- **Escalado:** desplazamiento configurable por etapa; por defecto, $\lfloor \log_2 L/2 \rfloor + 1$ bits en la FFT y el resto en la IFFT.

La IP de Xilinx tiene un modelo C bit-accurate, pero con más de una muestra por ciclo cambia su aritmética y pierde el cambio de N en ejecución (PG109). Todo lo que no sale de una fuente citada es estimación o diseño propio, y lo marco así.

Introducción: radix-2² en hardware

Esta sección explica las piezas que usa el resto del documento, de lo más abstracto a lo más concreto:

1. El grafo de puntos: qué se calcula.
2. La etapa R2: qué pasa cuando $\log_2 L$ es impar.
3. BF2I y BF2II por dentro: cómo se construye cada etapa.
4. Una traza ciclo a ciclo: cómo corre.
5. Los carriles: cómo se reparte cuando entran varias muestras por ciclo.

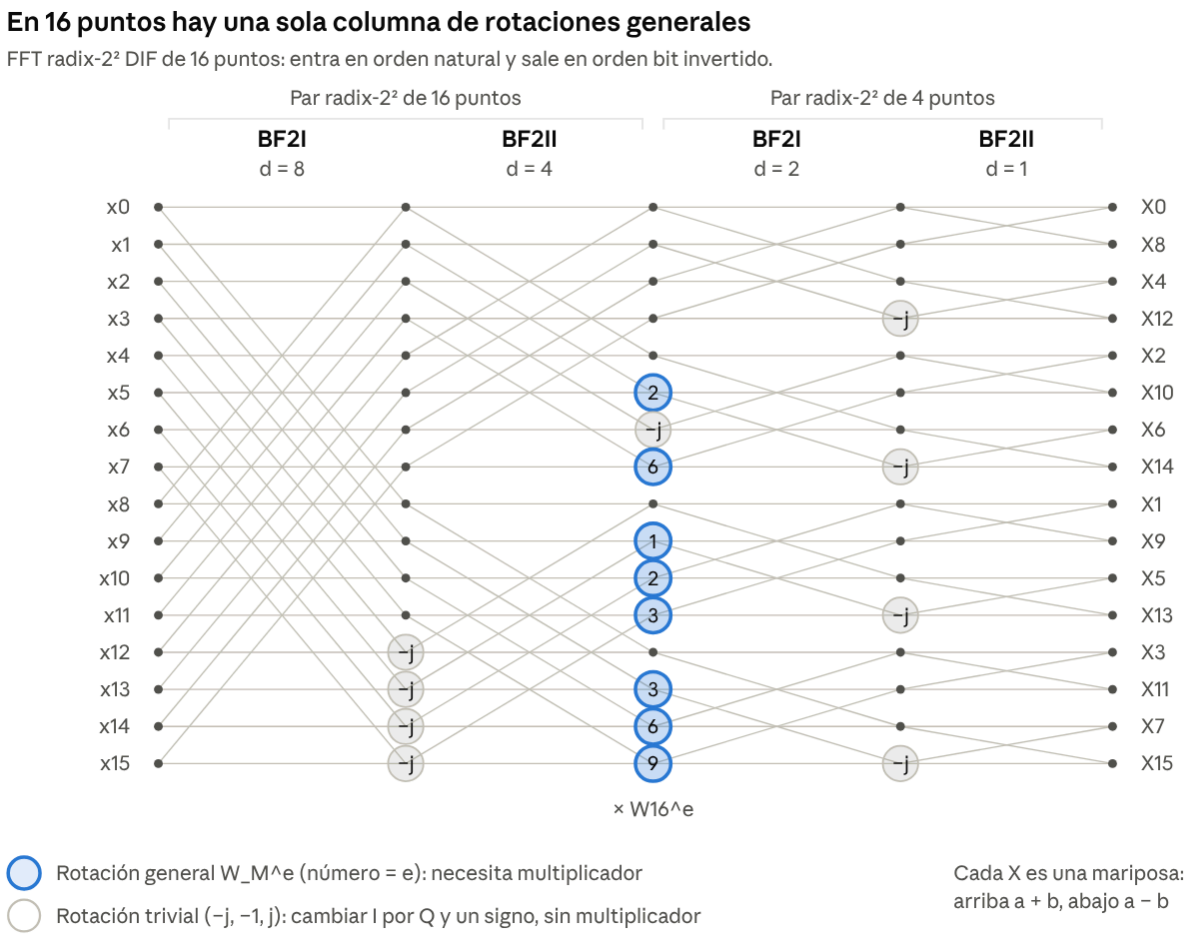
La referencia para ampliar es el survey de Garrido (§2.1 para el grafo, §4.1 para SDF y §5.1 para carriles en paralelo).

Tres palabras se repiten en todo el documento:

- **Mariposa:** toma dos valores a y b y saca a + b y a − b.
- **Distancia d:** cuántas posiciones separan a las dos muestras de una mariposa.
- **Twiddle o rotación:** un factor W_M^e = e^(−j2πe/M) que multiplica entre etapas. Es **trivial** si vale 1, −j, −1 o j, porque se aplica cambiando I por Q y signos, y **general** en los demás casos, donde hace falta un multiplicador.

El grafo de puntos

Radix-2^2 agrupa las etapas de a dos. Cada par tiene una mariposa de **tipo I (BF2I)** y una de **tipo II (BF2II)**; los nombres son de He y Torkelson (1996).



grafo de flujo radix-2² DIF · 16 puntos

- **BF2I:** mariposa común a distancia M/2, donde M es el tamaño de la sub-FFT. **No tiene rotación después.** En radix-2 común, ahí irían rotaciones generales de $W16^1$ a $W16^7$.
- **BF2II:** la misma mariposa a distancia M/4, pero **antes** multiplica por −j el último cuarto de cada bloque (x12 a x15). Es lo que queda de las rotaciones que radix-2 ponía entre las dos etapas, reducido a algo trivial.
- **Después del par:** la única rotación general, W16^e con $e = n_3 \cdot (k_1 + 2k_2)$. En la posición 6, e = 4 y $W16^4 = -j$, que es trivial.
- **Final:** el mismo par se repite sobre sub-FFT de 4 puntos, donde ya no hay rotaciones generales.

En hardware, cada columna de mariposas es una sola mariposa física y cada columna con círculos resaltados es un solo multiplicador. En 16 puntos, radix-2² tiene una columna de rotaciones generales y radix-2 tendría dos. Para L = 4096 son 5 contra 10.

De dónde salen los twiddles entre pares

El twiddle entre pares, $\mathbf{e = n_3 \cdot (k_1 + 2k_2)}$, sale de escribir los índices de la DFT en bits y separar la exponencial en pedazos. La derivación es para una sub-FFT de M puntos, que es lo que procesa cada par.

1. Partir los índices. La DFT de M puntos es X[k] = Σₙ x[n]·W_M^(n·k), con W_M = e^(−j2π/M). Radix-2² escribe:

```
n = \tfrac{M}{2}\,n_1 + \tfrac{M}{4}\,n_2 + n_3, \qquad k = k_1 + 2k_2 + 4k_3
```

Acá n_1, n_2, k_1 y k_2 valen 0 o 1, y n_3 y k_3 van de 0 a M/4 − 1. Es decir, n_1 y n_2 son los dos bits más altos de n, y k_1 y k_2 los dos más bajos de k.

2. Multiplicar n·k. Con W_M^M = 1, W_M^(M/2) = −1 y W_M^(M/4) = −j, cada producto queda así:

Producto	Factor	Qué es en el hardware
$(M/2)\cdot n_1\cdot k_1$	$(-1)^{(n_1k_1)}$	BF2I: mariposa a distancia M/2
$(M/2)\cdot n_1\cdot 2k_2$, $(M/2)\cdot n_1\cdot 4k_3$, $(M/4)\cdot n_2\cdot 4k_3$	1	Desaparecen (múltiplos de M)
$(M/4)\cdot n_2\cdot k_1$	$(-j)^{(n_2k_1)}$	El −j antes de BF2II
$(M/4)\cdot n_2\cdot 2k_2$	$(-1)^{(n_2k_2)}$	BF2II: mariposa a distancia M/4
$n_3\cdot(k_1 + 2k_2)$	$W_M^{(n_3(k_1+2k_2))}$	El twiddle entre pares
$n_3\cdot 4k_3$	$W_{(M/4)}^{(n_3k_3)}$	La FFT siguiente, de M/4 puntos (el próximo par)

Juntando, de adentro hacia afuera, queda la cadena del hardware: BF2I, el −j, BF2II, el twiddle y el par siguiente.

```
X[k_1+2k_2+4k_3] = \sum_{n_3} W_{M/4}^{n_3k_3}\, W_M^{n_3(k_1+2k_2)}
\sum_{n_2} (-1)^{n_2k_2}(-j)^{n_2k_1} \sum_{n_1} (-1)^{n_1k_1}\, x[n]
```

3. Pasarlo a posiciones del flujo. Las mariposas DIF reemplazan n_1 por k_1 y n_2 por k_2 en su lugar, así que después del par el dato de la posición q del bloque es $q = (M/2)\cdot k_1 + (M/4)\cdot k_2 + n_3$.

- El bit más alto de q es k_1, el segundo es k_2 y los de abajo son $n_3 = q \bmod (M/4)$.
- k_1, el bit alto, pesa 1 y k_2 pesa 2: el factor es esos dos bits **dados vuelta**. Por eso, en los cuatro cuartos del bloque, los factores salen 0, 2, 1 y 3.
- El −j aplica cuando $n_2 = 1$ y $k_1 = 1$, es decir, en el último cuarto del bloque ($q \geq 3M/4$).

Ejemplo con M = 16: e en W16^e para cada posición. Los valores 0, 4, 8 y 12 son triviales (1, −j, −1 y j); los demás necesitan multiplicador.

Cuarto del bloque	k_1, k_2	Factor	$n_3 = 0$	$n_3 = 1$	$n_3 = 2$	$n_3 = 3$
q = 0–3	0, 0	×0	0	0	0	0
q = 4–7	0, 1	×2	0	2	4 (−j)	6
q = 8–11	1, 0	×1	0	1	2	3
q = 12–15	1, 1	×3	0	3	6	9

Son los mismos números de la columna de rotaciones del grafo de 16 puntos.

4. Por qué ahorra multiplicadores. Radix-2 común, después de su primera etapa, multiplica por W_M^((M/4)·n_2 + n_3). Radix-2² parte ese factor en dos:

- **(−j)^(n_2k_1):** es trivial y se adelanta hasta BF2II.
- **W_M^(n_3k_1):** se posterga hasta después de la segunda etapa y se junta con el twiddle que radix-2 pondría ahí, W_M^(2n_3k_2).

Así, dos columnas de multiplicadores pasan a ser una.

5. Para L = 4096 y para la R2.

- **En cada par:** la misma fórmula con su M, aplicada a q mod M. La posición A usa M = 4096 y n_3 de 0 a 1023; la B usa M = 1024, repetida en cada bloque de 1024; y así hasta la E, con M = 16. El par de 4 puntos no tiene twiddle porque ahí n_3 = 0 siempre.
- **En la R2:** es la descomposición radix-2 común, n = (M/2)·n_1 + n′ y k = k_1 + 2k′. Sale un único twiddle, W_M^(n′·k_1), que en posiciones es e = q − M/2 para la mitad de abajo.

6. Cómo se genera en hardware. Hay dos formas, y dan los mismos bits porque la tabla de valores es la misma:

1. **Calcularlo:** separar del contador los dos bits altos (k_1, k_2) y los bajos (n_3) y hacer n_3·c con c entre 0 y 3. Multiplicar por 2 es un desplazamiento y por 3 es n_3 + 2n_3: un sumador chico, sin DSP.
2. **Guardarlo:** una ROM con el twiddle ya resuelto para cada posición, que es lo que propone este documento.

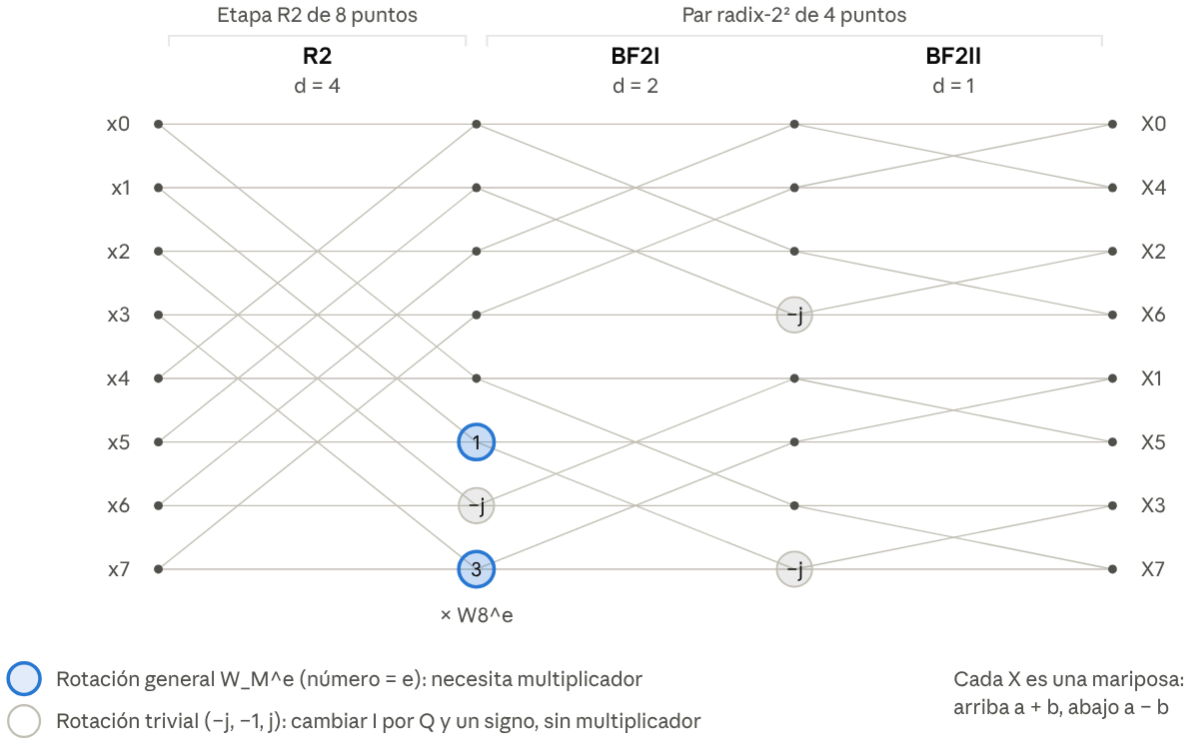
En los dos casos se usa una sola tabla de W4096: W_M^e = W4096^(e·4096/M), y como 4096/M es potencia de 2, el cambio de escala es otro desplazamiento.

La etapa R2

Cuando $\log_2$L es impar (8, 512, 2048), las etapas no se pueden agrupar todas de a dos y sobra una. Esa etapa suelta es la **R2**: una etapa radix-2 DIF común.

Con $\log_2 L$ impar, una etapa R2 va primero

FFT de 8 puntos: R2 con sus rotaciones W8^e y después el mismo par de 4 puntos que cierra la de 16.



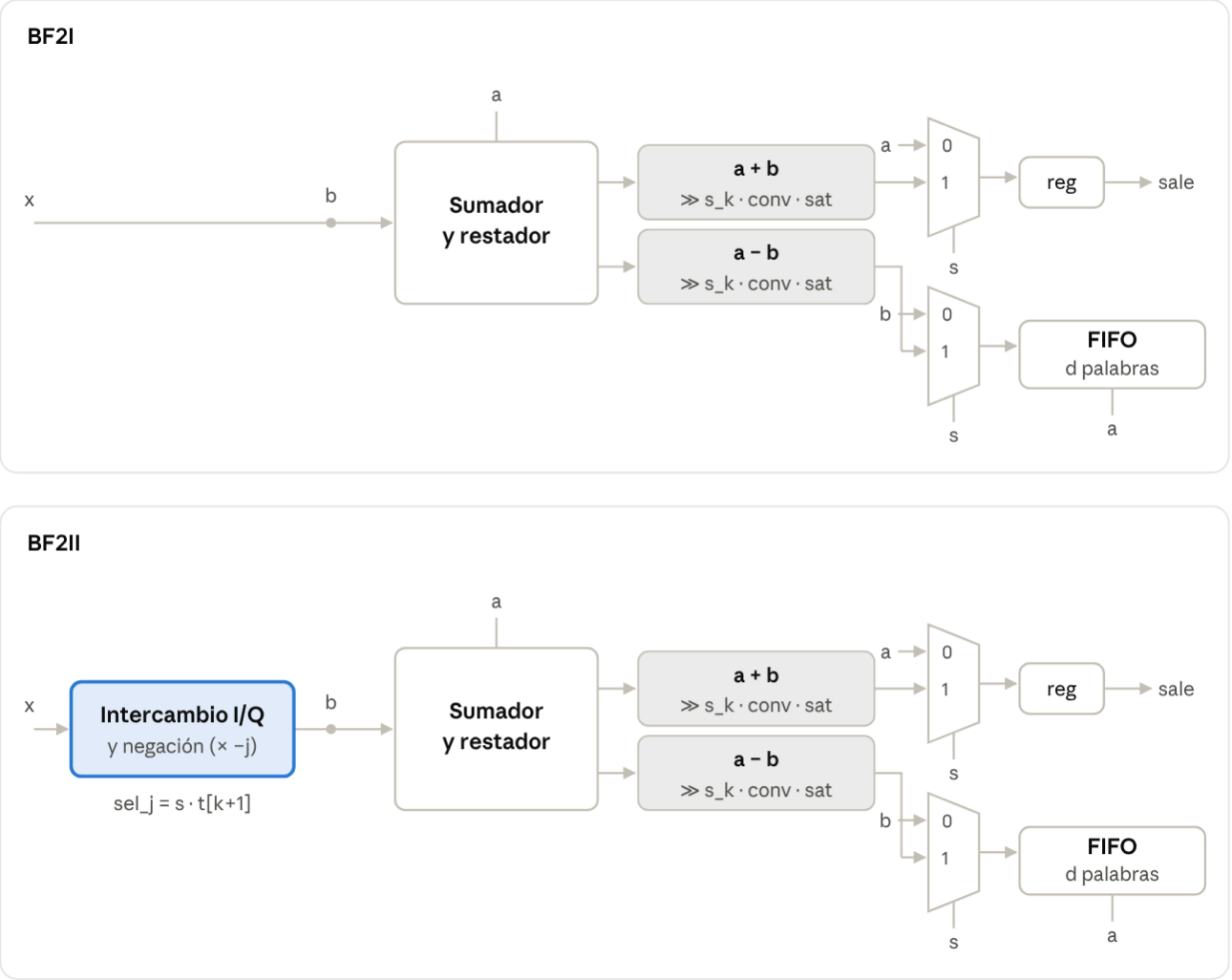
grafo de flujo con etapa R2 · 8 puntos

- **Qué hace:** la mariposa a distancia M/2 y, **después**, la rotación W_M^e sobre la mitad de abajo, con e = q − M/2. En 8 puntos, $W8^1$ y $W8^3$ son generales y $W8^2 = -j$ es trivial.
- **Por qué va primero:** así lo que queda después es exactamente la FFT de L/2. En 8 puntos es el mismo par de 4 puntos que cierra la FFT de 16, y en 2048 es la cadena entera de 1024. En esto se basa el cambio de N en ejecución salteando etapas (Arquitectura).

BF2I y BF2II por dentro

En la arquitectura SDF, cada etapa es una mariposa física que procesa una muestra por ciclo, con una FIFO de d palabras como memoria. Las dos etapas tienen el mismo núcleo; BF2II solo le agrega un bloque de entrada.

BF2I y BF2II por dentro: un sumador-restador, dos multiplexores y una FIFO
Una etapa SDF con FIFO de d palabras. BF2II es BF2I más el bloque resaltado que aplica −j antes de la mariposa.



Las letras a, b y s son conexiones: a es la salida de la FIFO; s = t[k], con k = $\log_2 d$, es un bit del contador del bloque.
s = 0: sale lo guardado (a) y entra b a la FIFO. s = 1: sale a + b y entra a − b. Bloques grises: desplazamiento, redondeo y saturación.

BF2I y BF2II a nivel de registros · etapa SDF

- **Sumador y restador:** calculan a + b y a − b en paralelo, en I y en Q por separado. a es el dato que sale de la FIFO y b el que entra. Son 4 sumadores reales de W_d + 1 bits, en lógica, sin DSP.
- **Desplazamiento, redondeo y saturación:** cada resultado se desplaza s_k bits (el escalado de esa etapa), se redondea con AC_RND_CONV y se satura con AC_SAT. Como se redondea antes de los multiplexores, la FIFO guarda palabras de W_d bits.
- **Multiplexor de salida:** con s = 0 saca a, la resta guardada del par anterior; con s = 1 saca a + b.
- **Multiplexor de la FIFO:** con s = 0 guarda b, la muestra que acaba de entrar; con s = 1 guarda a − b, que va a salir d ciclos después.
- **FIFO:** d palabras de 2·W_d bits (I y Q). Para d chico se hace con registros o SRL, y para d grande con BRAM.

- **Bloque −j (solo BF2II):** un multiplexor que deja pasar la muestra o la cambia por (Q, −I), que es multiplicar por −j. Se activa cuando s = 1 y el bit siguiente del contador vale 1, el último cuarto de cada bloque.
- **Registro de salida:** separa la etapa de la siguiente para cerrar timing.

Después de un BF2II, si hay rotación general, sigue el rotador: un multiplicador complejo de 3 DSP con su ROM de twiddles, direccionada por el mismo contador. Una R2 es este mismo circuito sin el bloque −j y siempre seguida de su rotador.

Señal	De dónde sale	Qué controla
t	Contador de $\log_2 L$ bits, posición dentro del bloque	Todo lo demás
s	Bit t[k], con $k = \log_2 d$	Los dos multiplexores: llenar o calcular
sel_j	$s \cdot t[k+1]$, solo en BF2II	El bloque −j
Dirección de ROM	t retrasado la latencia de la etapa	El twiddle del rotador que sigue
Modo L	Registro de configuración	Si la etapa se usa o se saltea (pasa el dato sin tocarlo)

Una FFT de 4 puntos ciclo a ciclo

Con x = [1, 2, 3, 4] entrando de a una muestra por ciclo, un BF2I con d = 2 seguido de un BF2II con d = 1 entrega X en orden bit invertido. Cada etapa tarda d ciclos y la salida empieza en el ciclo 3.

Ciclo	Entra	BF2I (t, fase)	BF2I guarda / saca	BF2II (t, fase)	BF2II guarda / saca
0	x0 = 1	0, llenar	guarda 1	—	—
1	x1 = 2	1, llenar	guarda 2	—	—
2	x2 = 3	2, calcular	saca 1 + 3 = 4, guarda −2	0, llenar	guarda 4
3	x3 = 4	3, calcular	saca 2 + 4 = 6, guarda −2	1, calcular	saca **X0 = 10**, guarda −2
4	x′0	0, llenar	saca −2	2, llenar	saca **X2 = −2**, guarda −2
5	x′1	1, llenar	saca −2	3, calcular, −j activo	entra (−2)·(−j) = 2j; saca **X1 = −2 + 2j**, guarda −2 − 2j
6	x′2	2, calcular	(bloque siguiente)	0, llenar	saca **X3 = −2 − 2j**

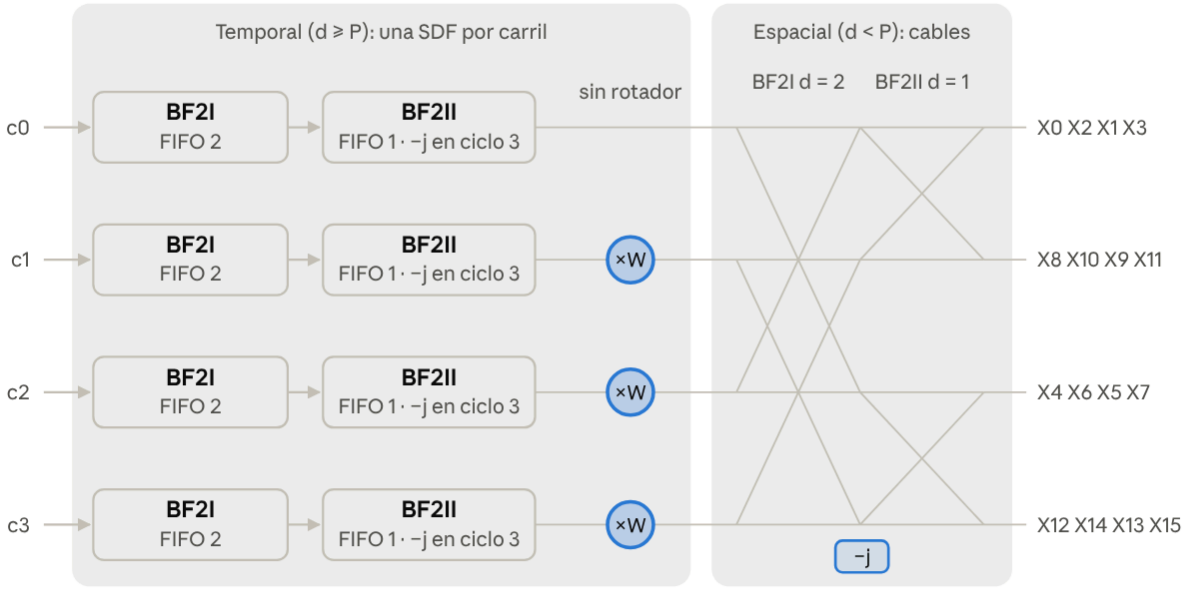
- **Latencia:** cada etapa entrega su primer dato d ciclos después. Para L = 4096 la suma de las FIFO es 2048 + 1024 + … + 1 = 4095 ciclos, más el pipeline.
- **Sin huecos:** el bloque siguiente entra mientras sale el anterior.
- **El −j cae siempre en la misma posición:** en BF2II es t = 3, la última de cada grupo de 4.
- **El modelo bit a bit no ve las FIFO ni el contador:** solo calcula los mismos valores que el grafo de puntos.

Con P carriles

Con P muestras por ciclo, la muestra j entra por el carril j mod P en el ciclo j div P. Lo que decide cómo se arma cada etapa es dónde cae la pareja de cada mariposa.

Con 4 carriles: dos etapas por carril y dos de cables

L = 16, P = 4. La muestra j entra por el carril j mod 4 en el ciclo j div 4; la salida sale en orden bit invertido.



En la zona espacial el −j cae siempre en el carril 3: es un cable cruzado, sin multiplexor. El carril 0 no rota porque siempre le toca $W^0 = 1$.

FFT de 16 puntos con 4 carriles · zona temporal y espacial

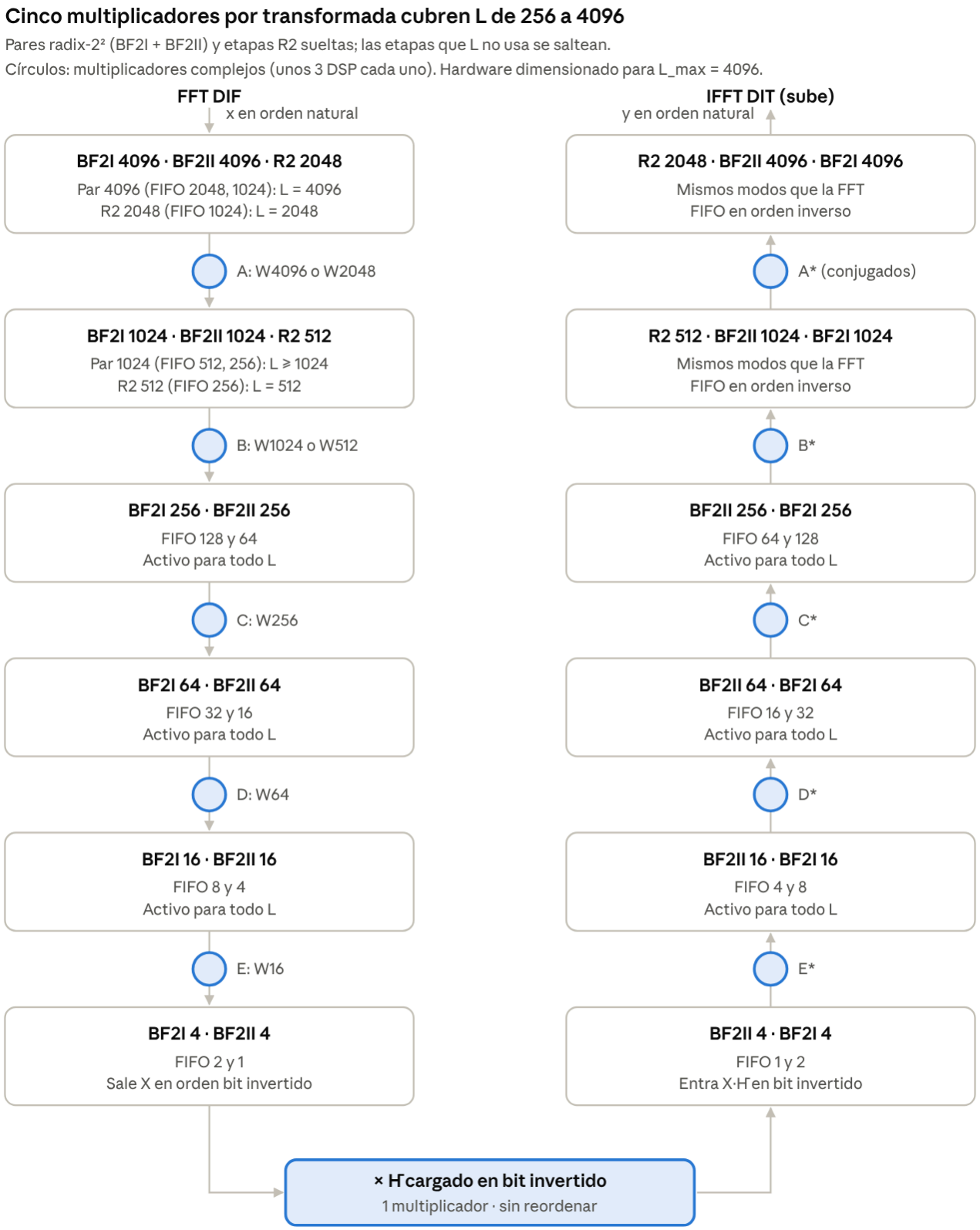
- **Zona temporal (d ≥ P):** la pareja (j, j + d) cae en el **mismo carril**, d/P ciclos después. Cada carril es una copia de la SDF de arriba, con FIFO de d/P. La memoria total sigue siendo del orden de L; lo que se multiplica por P son las mariposas y los rotadores.
- **Zona espacial (d < P):** la pareja llega en el **mismo ciclo**, por dos carriles distintos. Alcanza con una mariposa cableada entre esos carriles, sin FIFO ni multiplexores de fase.
- **Todo lo que depende de la posición queda fijo por carril en la zona espacial.** El −j cae siempre en el mismo carril, y un rotador que en ese carril ve un solo twiddle pasa a ser una constante (sumas fijas, sin DSP).

Esto es lo que explica la tabla de rotadores de la sección Estructura con P carriles. En la ZCU208 con P = 8 y L = 4096, la zona temporal son las etapas con d de 2048 a 8 más las dos R2, y la espacial son d = 4, 2 y 1. El rotador E (W16) cae justo en el borde: por eso quedan 2 rotadores generales y 4 constantes. Con P = 16, E queda entero en la zona espacial y no usa DSP.

Cualquier P da los mismos bits que la SDF de un carril, porque calcula los mismos nodos del grafo con los mismos redondeos; solo cambia en qué carril y en qué ciclo.

Arquitectura

La FFT es una cadena de 6 grupos de etapas con un multiplicador entre grupo y grupo. La IFFT es la misma cadena recorrida al revés, con los twiddles conjugados.



cadena FFT DIF → ×H̄ → IFFT DIT · L_max = 4096

Los dos primeros grupos combinan un par radix-2^2 y una etapa radix-2 suelta (R2) que nunca se usan a la vez. Por eso comparten el multiplicador que sigue: con L par el multiplicador lee la tabla del par y con L impar, la de la R2. En total son 5 multiplicadores por transformada para cualquier L. Con L = 256 solo trabajan C, D y E (diseño propio, verificado en Verificación hecha).

- **P = 1 (SDF):** cada etapa es una mariposa con una FIFO de realimentación, como en el survey de Garrido (§4.1). La FIFO de la IFFT DIT crece de 1 a L/2, al revés que en la FFT.
- **P > 1 (carriles en paralelo):** entran P muestras consecutivas por ciclo. Las etapas con distancia d ≥ P son P SDF en paralelo, con FIFO de d/P por carril. Las etapas con d < P combinan carriles del mismo ciclo, con mariposas cableadas y sin memoria. Corresponde a la familia MDF del survey (§5.1); en la ZCU208 se usa con P = 8 o 16 (Estructura con P carriles).
- **Condición para los mismos bits:** que cualquier P calcule cada nodo del grafo con el mismo twiddle, el mismo desplazamiento y el mismo redondeo. La arquitectura solo cambia en qué ciclo y en qué carril se calcula cada nodo.

Descomposición por etapas

La FFT de L puntos tiene $\log_2 L$ etapas radix-2 DIF, agrupadas así: si $\log_2 L$ es impar, **primero** va una etapa R2 suelta; después, pares BF2I + BF2II hasta llegar a 4 puntos. Esto hace que la cadena de un L chico sea exactamente la cola de la cadena de L_max.

L	Etapas (tipo, distancia d)	Rotaciones caras después de
256	I 128 · II 64 · I 32 · II 16 · I 8 · II 4 · I 2 · II 1	II 64, II 16, II 4 (3)
512	R2 256 · (cadena de 256)	R2, II 64, II 16, II 4 (4)
1024	I 512 · II 256 · (cadena de 256)	II 256 + las de 256 (4)
2048	R2 1024 · (cadena de 1024)	R2 + las de 1024 (5)
4096	I 2048 · II 1024 · (cadena de 1024)	II 1024 + las de 1024 (5)

Cada etapa es (tipo, d, M), con M el tamaño de la sub-FFT: M = 2d en I y R2, y M = 4d en II. Los datos entran en orden natural, una ventana de L muestras por bloque. Para la posición j del bloque se define q = j mod M.

1. **Solo en BF2II, antes de la mariposa:** se multiplica por −j (cambio de I y Q más negación, exacto) si q ≥ 3M/4.

2. **Mariposa:** para (j mod 2d) < d, sale a + b en j y a − b en j + d, donde a es el dato de la posición j y b el de j + d. Después se aplica el desplazamiento de esa etapa (Aritmética bit a bit).
3. **Rotación después de la etapa**, por W_M^e con W_M = e^(−j2π/M):
   - Después de R2: e = q − d si q ⩾ d; si no, nada.
   - Después de BF2II: e = $n_3 \cdot (k_1 + 2k_2)$, con $k_1 = \lfloor q/(M/2) \rfloor$, $k_2 = \lfloor q/(M/4) \rfloor \bmod 2$ y $n_3 = q \bmod (M/4)$.
   - Después de BF2I: nada.
4. **Rotaciones triviales:** si e mod (M/4) = 0, el factor es 1, −j, −1 o j y se aplica exacto, sin multiplicador ni redondeo. Solo las demás pasan por el multiplicador.

Ejemplo, L = 16 después del primer BF2II: e = [0 0 0 0 | 0 2 4 6 | 0 1 2 3 | 0 3 6 9]. El 4 es −j (trivial).

IFFT (DIT): las mismas etapas en orden inverso. En cada una, primero la rotación con el twiddle **conjugado**, después la mariposa, y en BF2II, después de la mariposa, ×(+j) donde q ⩾ 3M/4. Entra en orden bit invertido y sale en orden natural, sin el factor 1/L.

Producto por H̄: en la posición p del flujo se multiplica por H̄[bitrev(p)]. El host carga la tabla ya permutada.

Tabla de twiddles: W_M^e = W_4096^(e·4096/M), así que alcanza una sola tabla de L_max entradas (o un cuarto de onda con simetrías, si da los mismos valores). La genera un script y la leen tanto el RTL (inicialización de la ROM) como HALCON. Es diseño propio, verificado contra numpy en Verificación hecha.

Aritmética bit a bit

Hay un solo ancho de datos W_d en toda la cadena, con 1 bit de guarda. Se redondea en dos tipos de lugar: en la mariposa, cuando hay desplazamiento, y en cada multiplicación no trivial. Todo lo demás es exacto. Los valores por defecto son una propuesta mía, respaldada por la simulación de Verificación hecha; los valores para la ZCU208 están en la tabla siguiente.

Señal	Formato (I y Q por separado)	Valor para la ZCU208
Entrada x	`ac_fixed<W_in, 1, true>`, rango [−1, 1)	W_in = 14 (los ADC)
Datos internos (FFT, ×H̄, IFFT)	`ac_fixed<W_d, 2, true>`, rango [−2, 2)	W_d = 20

Señal	Formato (I y Q por separado)	Valor para la ZCU208
Twiddles	`ac_fixed<W_t, 1, true>`	W_t = 18
H̄	`ac_fixed<W_h, 1, true>`	W_h = 18
Salida y	`ac_fixed<W_out, 2, true>`, con AC_RND_CONV y AC_SAT	W_out = 14 (los DAC)

- **Por qué W_d = 20:** con 18 bits la FFT agrega tanto error como la propia entrada de 14 bits; con 20 agrega menos de la mitad (Verificación). No cuesta DSP: los datos van al puerto A, de 27 bits, del DSP48E2 (DS889). Solo cuesta un 11 % más de memoria.
- **Por qué no más de 20:** la salida se vuelve a cuantizar a 14 bits para el DAC, así que la mejora se pierde.
- **Límite de 18 bits en twiddles y H̄:** van al puerto B, de 18 bits. Con más, cada producto real pasaría a usar 2 DSP.

La entrada pasa a W_d por extensión de signo, que es exacta.

Puntos de cuantización, en orden dentro de una etapa:

1. **×(−j) en BF2II:** exacto. La única excepción es negar el mínimo (−2), que satura a 2 − 2^−(W_d−2).
2. **Mariposa:** a ± b se calcula exacto con W_d + 1 bits, se desplaza s bits a la derecha, se redondea a W_d con **AC_RND_CONV** y se satura con **AC_SAT**. Con s = 0 solo hay saturación.
3. **Rotación no trivial:** el producto complejo se calcula **exacto** y cada componente se redondea **una sola vez** a W_d, con **AC_RND** y AC_SAT. Como no hay redondeo intermedio, da los mismos bits con 3 o con 4 DSP.
4. **×H̄:** igual que el punto 3.
5. **Etapas salteadas** (las que L no usa): no redondean ni desplazan; pasan el dato tal cual.

Por qué dos modos de redondeo: al desplazar 1 bit, la mitad de los valores caen justo en un empate. Con AC_RND eso sumaría un sesgo medio de +¼ LSB por etapa, que se acumula como continua. AC_RND_CONV lo evita, y el DSP48E2 lo implementa con el detector de patrón (DS889). En los productos los empates son raros, así que AC_RND, que es una constante sumada en el DSP, alcanza. Los modos quedan como parámetros.

Tablas: twiddles y H̄ se redondean con AC_RND a su formato Q1. El valor +1 no es representable y satura al máximo. Nunca se necesita +1 exacto, porque las rotaciones triviales no pasan por la tabla.

Escalado: es un vector de desplazamientos por etapa, uno para la FFT (s_f) y otro para la IFFT (s_i), que carga el host junto con N. Como |H| = 1 y el bloque no debe cambiar la potencia, la suma de los desplazamientos de FFT e IFFT tiene que ser $\log_2 L$. Con esa condición la ganancia total es 1.

- **Por defecto ("mitad + 1"):** en la FFT, s = 1 en cada etapa BF2I y R2, y también en la segunda etapa. Total: $\lfloor \log_2 L/2 \rfloor + 1$ bits. En la IFFT, cada etapa desplaza lo complementario (s_i = 1 − s_f), lo que suma el resto.
- **Por qué:** para una señal con espectro parecido a ruido, como QAM con RRC, el valor eficaz crece en $\sqrt{L}$ dentro de la FFT. Desplazar la mitad de las etapas lo mantiene constante, y el bit extra cubre los picos espectrales. Sin ese bit aparecieron saturaciones con L = 256 y 4096 (Verificación hecha).
- **Conservador** (todo el desplazamiento en la FFT): nunca satura, pero pierde unos $\log_2 L/2$ bits de SNR. Con 16 bits llega a un NRMSE de $6{,}6 \cdot 10^{-3}$ para L = 4096.
- **Sin escalado en la FFT** (como Jiang): con un datapath de W_d bits no alcanza. Satura masivamente salvo que se reduzca la entrada, como hace Jiang con 10 bits sobre 18.
- **Bloque flotante:** no lo recomiendo. Agrega un exponente por bloque que cambia la ganancia bloque a bloque y complica el modelo, sin necesidad.

Propongo agregar un **contador de saturaciones** por transformada que el host pueda leer. El modelo lo reproduce exactamente y sirve para elegir el vector de escalado.

Consideraciones de la ZCU208

Acá empieza la implementación en la placa. El recurso escaso es el DSP; la lógica y la memoria sobran para la CD. Los datos del sistema son los que dio el equipo; los recursos salen de DS889 y del kit (Mouser).

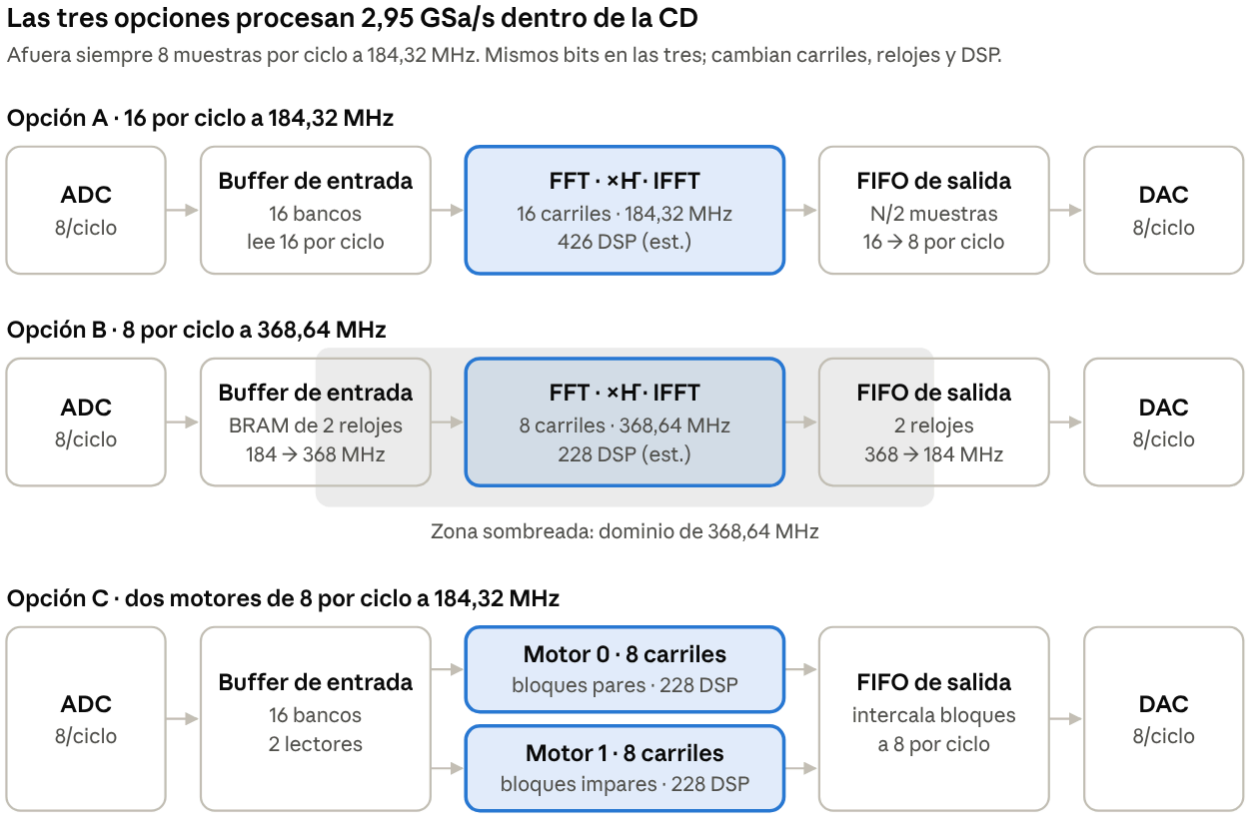
Aspecto	Valor	Qué implica para la CD
Dispositivo	XCZU48DR (Gen 3)	—
DSP48E2	4 272, multiplicador 27 × 18	Twiddles y H ≤ 18 bits; datos hasta 27 bits sin costo extra
CLB LUT	425 280	Margen para sumas fijas y CORDIC acotados
Block RAM	1 080 BRAM36 (38 Mb)	La CD usa del orden de 60 a 70 (est.)
UltraRAM	80 bloques (22,5 Mb)	No hace falta para la CD
Convertidores	8 ADC y 8 DAC de 14 bits	W_in = W_out = 14

Aspecto	Valor	Qué implica para la CD
Paralelismo del sistema	P = 8 muestras por ciclo	Todos los bloques reciben 8 por ciclo
Reloj del sistema	184,32 MHz (período 5,43 ns)	8 × 184,32 MHz = 1,47456 GSa/s

- **Flujo continuo:** los ADC y DAC no esperan, así que no hay backpressure. La CD tiene que aceptar y entregar exactamente 8 muestras por ciclo, siempre (Flujo continuo).
- **Presupuesto compartido:** el ZU48DR aloja también ISI genérica, AWGN, PMD, offset de portadora, ruido de fase del láser, error de temporización y la lógica de los convertidores. Con los números de Fiber-on-Chip (Tabla I), solo PMD, AWGN y ruido de fase costarían unos 250 DSP por carril, unos 2 000 con P = 8 (estimación de orden de magnitud).
- **Frecuencia máxima:** DS889 no da la f_clk máxima de la lógica programable ni el grado de velocidad del kit. Para la opción B hace falta la hoja AC/DC del dispositivo.
- **Reloj doble:** 368,64 MHz = 2 × 184,32 MHz se puede sacar del mismo MMCM, en fase con el reloj del sistema. El cruce entre dominios es síncrono.
- **f_s de H:** el host tiene que calcular $\bar{H}$ con la f_s **emulada** (64, 72 o 120 GSa/s). Con la tasa real de 1,47 GSa/s, R valdría 1 incluso a 2000 km, porque R crece con f_s^2. El hardware corre la misma física escalada en el tiempo, como en Fiber-on-Chip.

Propuestas de arquitectura para la ZCU208

La CD tiene que procesar **2,95 GSa/s internamente** para sostener los 1,47456 GSa/s del sistema. El overlap-save al 50 % hace que la FFT vea cada muestra dos veces ($R_{CD} = P \cdot f_{clk}/2$), y con 8 muestras por ciclo a 184,32 MHz se llegaría solo a 0,74 GSa/s. Hay tres formas de duplicar el caudal del núcleo sin tocar el resto del sistema:



opciones A, B y C en la ZCU208 · ADC → buffer → núcleo → FIFO → DAC

En las tres, el núcleo (FFT, ×Ĥ e IFFT) es la cadena radix-2^2 de la sección Arquitectura, con P carriles. Los recuadros resaltados son los que gastan DSP.

Comparación de las opciones

La opción B cuesta la mitad de DSP que A o C, a cambio de un segundo dominio de reloj a 368,64 MHz. Las tres dan los mismos bits. Las cifras son estimaciones mías, para L_max = 4096:

- **DSP:** del conteo de rotadores carril por carril (Estructura con P carriles), a 3 DSP por multiplicador complejo general (dato de Jiang, Tabla II).
- **LUT y BRAM:** de orden de magnitud.

	A: 16 por ciclo a 184,32 MHz	B: 8 por ciclo a 368,64 MHz	C: 2 motores de 8 por ciclo a 184,32 MHz
Multiplicadores generales (FFT + IFFT + Ĥ)	63 + 63 + 16 = 142	34 + 34 + 8 = 76	2 × 76 = 152
DSP	426 (10,0 %)	228 (5,3 %)	456 (10,7 %)

	A: 16 por ciclo a 184,32 MHz	B: 8 por ciclo a 368,64 MHz	C: 2 motores de 8 por ciclo a 184,32 MHz
DSP con ×H en CORDIC	378 (8,8 %)	204 (4,8 %)	408 (9,6 %)
LUT	~55 000 (13 %)	~30 000 (7 %)	~60 000 (14 %)
BRAM36	~70 (6 %)	~60 (6 %)	~120 (11 %)
Relojes	Uno	Dos (184,32 y 368,64 MHz)	Uno
Latencia con L = 4096	~640 ciclos + p, ~3,5 µs	~1 280 ciclos + p a 368,64 MHz, ~3,5 µs	~1 280 ciclos + p, ~6,9 µs
Riesgo principal	Ninguno especial; más DSP	Cerrar timing a 368,64 MHz	El doble de memoria y un despachador de bloques

Los porcentajes son sobre el ZU48DR. p son los registros de pipeline, que dependen del RTL.

- **A si a la CD le tocan unos 430 DSP o más:** es la más simple de cerrar.
- **B si el presupuesto es más ajustado:** cuesta la mitad, pero el equipo tiene que aceptar el segundo reloj y comprobar el timing temprano, con una síntesis de prueba de una etapa.
- **C solo si se quiere reutilizar exactamente el mismo módulo de 8 carriles que el resto del sistema:** no ahorra nada frente a A y gasta más memoria.

Llevar todas las rotaciones a CORDIC ya no conviene con tantos carriles: con 16 costaría unas 176 000 LUT, el 41 % del chip (estimación).

Estructura con P carriles

Con P muestras consecutivas por ciclo, la cadena de la sección Arquitectura (la misma de L_max = 4096, con las mismas etapas salteables) se reparte en dos zonas:

1. **Etapas temporales (d ≥ P):** las dos muestras de cada mariposa caen en el **mismo carril**, en ciclos distintos. Cada carril es una SDF con FIFO de d/P. Con P = 16 son las etapas de d = 2048 a 16, más las dos R2. Con P = 8 se suma la de d = 8.
2. **Etapas espaciales (d < P):** las dos muestras llegan en el **mismo ciclo**, en carriles distintos. Son mariposas cableadas entre carriles, sin memoria. Con P = 16 son las de d = 8, 4, 2 y 1; con P = 8, las de d = 4, 2 y 1.

La IFFT DIT es la misma estructura espejada: primero las etapas espaciales y después las temporales.

Rotadores por posición. En cada posición de multiplicador (A a E en el diagrama de Arquitectura) hay un rotador por carril. Según qué twiddles le tocan a cada carril, sumando todos los modos de L, el rotador es de uno de tres tipos:

- **Ninguno:** solo le tocan valores triviales.
- **Constante:** siempre el mismo valor; va con sumas fijas en LUT, sin DSP.
- **General:** varios valores; va con un DSP y una ROM de twiddles.

Conteo por FFT (la IFFT es igual), calculado con el modelo de Descomposición por etapas:

Posición	Twiddles	P = 8: generales	P = 8: constantes	P = 16: generales	P = 16: constantes
A	W4096 o W2048	8	0	16	0
B	W1024 o W512	8	0	16	0
C	W256	8	0	16	0
D	W64	8	0	15	0
E	W16	2	4	0	8
Total por FFT		**34**	**4**	**63**	**8**

Con P = 16 la posición E queda entera en la zona espacial: cada carril ve un solo twiddle y no hace falta ningún DSP. Sumando FFT, IFFT y los P multiplicadores de ×H̄ salen los 76 y 142 generales de la comparación.

Memoria por carril. Cada rotador general necesita su ROM con los twiddles que le tocan, así que con P grande las tablas se replican. En las posiciones A y B son de 256 a 1 024 entradas y van en BRAM; en C y D son chicas y entran en LUT. Es la parte que más crece con P (estimación).

Cuánto hardware queda sin usar según L

El hardware se dimensiona para L_max = 4096, así que con un L más chico los grupos del principio de la FFT (y los del final de la IFFT) quedan salteados y sin trabajar. Es el precio de poder cambiar N en ejecución sin resintetizar. La tabla es una estimación para la opción B (8 carriles, 228 DSP), con el mismo conteo de rotadores de arriba:

L	Grupos activos	Multiplicadores activos por carril	DSP en uso	DSP sin trabajar	FIFO en uso (por transformada)
4096	Todos (par de 4096)	A, B, C, D, E	228	0	4 095 palabras

L	Grupos activos	Multiplicadores activos por carril	DSP en uso	DSP sin trabajar	FIFO en uso (por transformada)
2048	Todos (R2 de 2048)	A, B, C, D, E	228	0	2 047
1024	Desde el par de 1024	B, C, D, E	180	48 (21 %)	1 023
512	Desde la R2 de 512	B, C, D, E	180	48 (21 %)	511
256	Desde el par de 256	C, D, E	132	96 (42 %)	255

- **Las R2 siempre sobran en parte:** con L par no se usan, pero son baratas (una mariposa en lógica y una FIFO; el multiplicador lo comparten con el par). Sus FIFO pueden compartir memoria con las del par del mismo grupo, porque nunca trabajan a la vez. Así la memoria total queda en unas 4 095 palabras por transformada, en lugar de 5 400.
- **El consumo no cambia con N:** se reserva siempre el máximo.
- **Si el presupuesto de DSP aprieta:** L_max puede ser también un parámetro de síntesis. Con el mismo RTL se sintetiza L_max = 4096 para los ensayos de larga distancia y L_max = 1024 o 256 para los de corta. En esas versiones los grupos de adelante no existen, y quedan libres hasta 96 DSP para otros bloques. El modelo bit a bit no cambia, porque la cadena de cada L es la misma en todas las versiones. El costo es resintetizar al cambiar de rango de distancias.

Detalle de cada opción

Las tres usan la cadena y la aritmética de la parte general; cambia cómo se reparten las muestras entre carriles y relojes. El modelo bit a bit es el mismo para las tres. Lo único que depende de la opción es el transitorio al cambiar N, cuántas muestras inválidas aparecen, y eso hay que definirlo para la opción elegida. Todo lo que sigue es diseño propio, con cifras estimadas para L_max = 4096.

Opción A: 16 muestras por ciclo a 184,32 MHz

1. **Buffer de entrada:** 16 bancos (banco = j mod 16) de 256 palabras de 28 bits (14 + 14). El ADC escribe 8 bancos por ciclo; la FFT lee los 16. Cada bloque lee [kN − N, kN + N), y como N ⩾ 128 siempre es múltiplo de 16.
2. **FFT de 16 carriles:**
   - Diez etapas temporales por carril, con FIFO de d/16: 128, 64, 64 (R2), 32, 16, 16 (R2), 8, 4, 2 y 1. Son 335 palabras por carril, unas 5 400 en total.

    - Cuatro etapas espaciales.
3. **×H̄:** 16 multiplicadores. H̄ con doble buffer, en 16 bancos de 2 × 256 palabras.
4. **IFFT:** la misma estructura espejada.
5. **Salida:** cada bloque entrega sus N muestras útiles en N/16 ciclos, de a 16 por ciclo. Una FIFO de N/2 muestras (1 024 con N = 2048) las pasa a exactamente 8 por ciclo para el DAC.
6. **Control:** contador de bloques, modo L (qué etapas se saltean y qué tablas usan A y B) y conmutación de H̄ en bordes de bloque.

A favor: un solo reloj y un timing cómodo (5,43 ns). En contra: es la de más DSP (426).

Opción B: 8 muestras por ciclo a 368,64 MHz

1. **Dominios:** ADC, DAC y resto del sistema a 184,32 MHz; núcleo de la CD a 368,64 MHz, del mismo MMCM.
2. **Buffer de entrada:** 8 bancos de 512 palabras en BRAM de doble puerto con relojes independientes. El puerto de escritura va a 184,32 MHz (8 por ciclo) y el de lectura a 368,64 MHz (8 por ciclo). **La propia RAM hace el cruce de dominio**, sin FIFO extra.
3. **FFT de 8 carriles:**
    - Once etapas temporales, con FIFO de d/8: 256, 128, 128 (R2), 64, 32, 32 (R2), 16, 8, 4, 2 y 1.
    - Tres etapas espaciales.
4. **×H̄:** 8 multiplicadores. **IFFT:** la estructura espejada.
5. **Salida:** FIFO de N/2 muestras, también de doble reloj. Se escribe en ráfagas a 368,64 MHz y se lee a 8 por ciclo a 184,32 MHz.
6. **Timing a 2,71 ns:**
    - Registro después de cada suma y cada multiplexor.
    - DSP48E2 con todos sus registros internos.
    - BRAM con registro de salida.
    - Señales de modo replicadas para bajar el fanout.

A favor: la mitad de DSP (228). En contra: el segundo reloj y el riesgo de timing. Conviene sintetizar temprano una etapa con rotador y FIFO en BRAM, para confirmar que cierra a 368,64 MHz en el grado de velocidad del kit.

Opción C: dos motores de 8 muestras por ciclo a 184,32 MHz

1. **Despachador:** los bloques pares van al motor 0 y los impares al motor 1. Cada motor recibe un bloque cada L/8 ciclos y lo procesa en ese tiempo.

2. **Buffer de entrada:** tiene que atender dos lecturas simultáneas de 8 por ciclo, así que se organiza en 16 bancos, igual que en A.
3. **Motores:** cada uno es el módulo de 8 carriles de B, pero a 184,32 MHz, con su propio $\times\bar{H}$ y sus ROM de twiddles.
4. **Salida:** se intercalan los bloques en orden y se pasan por una FIFO a 8 por ciclo.

A favor: un solo módulo de 8 carriles, igual al ancho del resto del sistema, y un solo reloj. En contra: más DSP que A (456), el doble de memoria y el doble de latencia (~6,9 µs).

Flujo continuo: orden, buffers, latencia y reconfiguración

Como el ADC y el DAC no se pueden detener, el bloque acepta y entrega exactamente 8 muestras por ciclo, siempre. No hay reordenamiento en ningún punto: la entrada y la salida van en orden natural, y el orden bit invertido queda encerrado entre la FFT y la IFFT.

- **Entrada:** con la opción A, leer un bloque de L muestras a 16 por ciclo dura lo mismo que tardan en llegar sus N nuevas a 8 por ciclo. El balance es exacto. Con la opción B pasa lo mismo, en el dominio de 368,64 MHz. Las N muestras nuevas se escriben sobre la mitad más vieja del buffer, que ya se leyó.
- **Salida:** la IFFT entrega L muestras por bloque; se descartan las primeras N y las N útiles salen en ráfagas. Una FIFO de unas N/2 muestras hace que el DAC reciba exactamente 8 por ciclo.
- **Latencia (est.):** con la opción A y L = 4096, unos 640 ciclos más el pipeline, alrededor de 3,5 µs, más la FIFO de salida. Las etapas salteadas no suman latencia. Aparte está el retardo algorítmico de N/2 muestras, que ya está en el modelo de referencia. La latencia en ciclos no afecta la comparación bit a bit, porque se comparan secuencias de muestras.

Cambio	Qué hace el hardware	Qué tiene que reproducir el modelo
Solo $\bar{H}$ (otra distancia, misma N)	$\bar{H}$ con doble buffer; conmuta en un borde de bloque, sin cortes	Los bloques desde k_0 usan el $\bar{H}$ nuevo; el hardware informa k_0
Vector de escalado	En un borde de bloque, igual que $\bar{H}$	Igual: desde k_0
N (y por lo tanto L)	No puede pausar la entrada: durante uno o dos bloques entrega muestras inválidas	Cuántas muestras inválidas y con qué valor (por ejemplo, ceros); después, `cd_overlap_save` desde cero

La cantidad de muestras inválidas y su valor se tienen que fijar con el equipo para la opción elegida, para que el modelo lo reproduzca exactamente.

Verificación hecha

La descomposición es exacta y, con 18 bits y el escalado por defecto, el error de cuantización queda entre $1{,}1 \cdot 10^{-4}$ y $1{,}6 \cdot 10^{-4}$ de NRMSE. Las simulaciones son mías, en Python.

Estructura (punto flotante):

- La FFT radix-2^2 DIF de Descomposición por etapas coincide con `np.fft.fft` en orden bit invertido, y la IFFT DIT coincide con L· `np.fft.ifft` partiendo de esa entrada. Probado para L = 4, 8, …, 4096, con error relativo máximo $\leqslant 6 \cdot 10^{-16}$.
- La cadena de cada L es la cola de la cadena de L_max: la de 1024 es la de 4096 sin sus dos primeras etapas, y la de 2048 es R2 más la de 1024.
- Las rotaciones caras dan 3, 4, 4, 5 y 5 columnas para L = 256 a 4096, que es la cantidad de multiplicadores de Arquitectura.

Punto fijo: el NRMSE de la salida del bloque cuantizado contra overlap-save en doble precisión sobre la **misma** entrada cuantizada. Mide solo el error interno.

- Señal: 16-QAM con 2 muestras por símbolo (64 GSa/s), RRC con β = 0,1, 20 000 símbolos, una semilla. Entrada de 10 bits con pico de componente 0,95.
- W_d = W_t = W_h en cada columna. Mariposas con AC_RND_CONV, productos con AC_RND, saturación en todos los puntos.

Escenario	Conservador, 16 bits	Conservador, 18 bits	Por defecto, 16 bits	Por defecto, 18 bits
200 km, R = 111, L = 256	$1{,}6 \cdot 10^{-3}$	$4{,}1 \cdot 10^{-4}$	$4{,}9 \cdot 10^{-4}$	$1{,}2 \cdot 10^{-4}$
1000 km, R = 559, L = 2048	$4{,}7 \cdot 10^{-3}$	$1{,}2 \cdot 10^{-3}$	$4{,}5 \cdot 10^{-4}$	$1{,}1 \cdot 10^{-4}$
2000 km, R = 1117, L = 4096	$6{,}6 \cdot 10^{-3}$	$1{,}7 \cdot 10^{-3}$	$6{,}5 \cdot 10^{-4}$	$1{,}6 \cdot 10^{-4}$

Ninguna de estas combinaciones saturó. Para comparar, el error propio del método es de $2 \cdot 10^{-4}$ a $1{,}6 \cdot 10^{-3}$ entre 100 y 2000 km, con 16-QAM a 2 muestras por símbolo.

- **Mitad sin el bit extra:** con L = 256 y L = 4096 hubo 3 y 5 saturaciones, que con L = 4096 dejan el NRMSE en $1{,}4 \cdot 10^{-3}$ aun con 18 bits.

- **Sin escalado en la FFT:** satura en cientos de miles de puntos, con NRMSE de 0,8 a 0,96.

Con los datos de la ZCU208 (entrada de 14 bits, twiddles y H̄ de 18, escalado por defecto), con 2 y 1,2 muestras por símbolo. Las columnas de W_d son el NRMSE contra la referencia en punto flotante de la misma entrada ya cuantizada:

Escenario	Error de cuantizar la entrada a 14 bits	Interno, W_d = 18	Interno, W_d = 20	Interno, W_d = 22
2 sps (64 GSa/s), 200 km, L = 256	$1{,}1\cdot10^{-4}$	$1{,}2\cdot10^{-4}$	$3{,}1\cdot10^{-5}$	$9{,}7\cdot10^{-6}$
2 sps (64 GSa/s), 2000 km, L = 4096	$1{,}1\cdot10^{-4}$	$1{,}6\cdot10^{-4}$	$4{,}1\cdot10^{-5}$	$1{,}3\cdot10^{-5}$
1,2 sps (72 GSa/s), 200 km, L = 512	$1{,}1\cdot10^{-4}$	$9{,}8\cdot10^{-5}$	$2{,}5\cdot10^{-5}$	$9{,}0\cdot10^{-6}$
1,2 sps (72 GSa/s), 2000 km, L = 4096	$1{,}1\cdot10^{-4}$	$1{,}6\cdot10^{-4}$	$4{,}2\cdot10^{-5}$	$1{,}3\cdot10^{-5}$

No hubo saturaciones en ningún caso, incluido 1,2 muestras por símbolo. De acá sale la elección de W_d = 20: es el primer ancho en que el error interno queda claramente por debajo del de la entrada.

Lo que falta probar: el término de β_3, 120 GSa/s, otras semillas y modulaciones, y la salida recortada a 14 bits. Cuando exista `quantization.py`, estos resultados tienen que reproducirse con él.

Respuestas a las preguntas del pedido

1. **¿IP de Xilinx o FFT propia?** Propia.
   - La IP en Pipelined Streaming tiene un modelo C bit-accurate listo para usar (PG109, *C Model*). Pero es una caja negra: no hay especificación por etapa para escribir en `ac_fixed`.
   - Con SSR > 1, que en la ZCU208 es obligatorio, cambia la aritmética (twiddles de 19 bits, otro redondeo) y pierde la configuración en ejecución de NFFT (PG109).
   - Su IFFT pide entrada en orden natural, lo que obliga a reordenar.
   - La propia se especifica nodo por nodo y da los mismos bits para cualquier P.
2. **¿Radix-2^2 o radix-2 con P > 1?** Radix-2^2. Para L = 4096 son 5 multiplicadores por transformada y carril contra 10 de radix-2, y los dos dan el mismo grafo con otras rotaciones. Radix-2^3 y radix-2^k reducen el costo de algunas rotaciones (Garrido), pero no la cantidad de multiplicadores generales en este rango.

3. **¿Cómo se cambia N en ejecución?** Salteando las primeras etapas de la FFT y las últimas de la IFFT. Los multiplicadores A y B cambian de tabla según L sea par o impar (Arquitectura). El modelo corre la cadena de L, porque una etapa salteada pasa el dato exacto. Lo que sí tiene que modelar es el transitorio de la reconfiguración (Flujo continuo).
4. **¿Hace falta reordenar?** No. La FFT DIF entrega bit invertido, $\bar{H}$ se carga ya permutado y la IFFT DIT acepta bit invertido y devuelve orden natural.
5. **¿Qué escalado?** Un vector de desplazamientos por etapa que suma $\log_2 L$ entre FFT e IFFT. Por defecto, $\lfloor \log_2 L/2 \rfloor + 1$ bits en la FFT y el resto en la IFFT. Que $|\bar{H}| = 1$ es lo que permite repartir así sin cambiar la potencia (Aritmética bit a bit).
6. **¿Buffer de entrada?** Una RAM de L_max = 4096 palabras complejas de 28 bits, repartida en 16 bancos (opción A) o en 8 bancos de doble reloj (opción B) (Detalle de cada opción).
7. **¿Qué copiar de Fiber-on-Chip?** Que el hardware vea solo parámetros normalizados: N, $\bar{H}$ y el escalado. f_s, D, λ y z los usa el host para calcular $\bar{H}$, con la f_s **emulada**. FoC corre a 100 MHz con un carril y expresa la física normalizada al período de muestra (DGD en fracciones de T_s, ruido de fase con ΔfT_s).
8. **Recursos y tasa:** para la ZCU208, en Comparación de las opciones: de 228 a 456 DSP según la opción, para 1,47456 GSa/s.
9. **Parámetros nuevos:** en la tabla siguiente.

Parámetros para `HardwareParams`

Se dividen en dos grupos: los que se fijan al sintetizar y los que se cargan en ejecución con cada configuración. Los nombres son una propuesta.

Parámetro	Cuándo se fija	Valor para la ZCU208	Para qué
`L_max` (o el `N_max` actual)	Síntesis	4096	Dimensiona etapas, FIFO y tablas
`P_sys`	Síntesis	8	Muestras por ciclo en la interfaz del bloque
`f_clk`	Síntesis	184,32 MHz	Reloj del sistema; R = P_sys·f_clk = 1,47456 GSa/s
`P_cd`	Síntesis	16 (opción A) u 8 (opción B)	Carriles del núcleo; no cambia los bits

Parámetro	Cuándo se fija	Valor para la ZCU208	Para qué
`clk_mult_cd`	Síntesis	1 (opción A) o 2 (opción B)	Reloj del núcleo = clk_mult_cd · f_clk
`W_in`	Síntesis	14	Ancho de la entrada, Q1
`W_d`	Síntesis	20	Ancho interno de datos, Q2
`W_t`	Síntesis	18	Ancho de twiddles, Q1 (⩽ 18 por el DSP48E2)
`W_h`	Síntesis	18	Ancho de H̄, Q1 (⩽ 18 por el DSP48E2)
`W_out`	Síntesis	14	Ancho de la salida al DAC
`rnd_butterfly`	Síntesis	AC_RND_CONV	Redondeo después del desplazamiento
`rnd_mult`	Síntesis	AC_RND	Redondeo de rotaciones y ×H̄
`ovf`	Síntesis	AC_SAT	Desborde en todos los puntos
`N`	Ejecución	según R	Elige la cadena de L = 2N
`Hbar`	Ejecución	del host, con la f_s emulada	L valores Q1, ya en orden bit invertido
`shift_fft`, `shift_ifft`	Ejecución	“mitad + 1”	Vectores de $\log_2 L$ desplazamientos que suman $\log_2 L$
`n_invalid`	A definir	según la opción	Muestras inválidas al cambiar N y su valor

El modelo también necesita reproducir dos salidas del hardware: el **contador de saturaciones** por transformada y el **índice de bloque k_0** en que entra en vigor un H̄ o un escalado nuevo.

Preguntas para el equipo

- [ ] ¿Cuántos DSP le tocan a la CD dentro del presupuesto de todo el emulador (ISI genérica, AWGN, PMD, offset de portadora, ruido de fase, error de temporización, ADC/DAC)?
- [ ] ¿Opción A (16 muestras por ciclo a 184,32 MHz) u opción B (8 a 368,64 MHz)? ¿El equipo acepta un segundo dominio de reloj?

- [ ] ¿Qué grado de velocidad tiene el XCZU48DR de la ZCU208? Con la hoja AC/DC se confirma si la opción B cierra a 368,64 MHz.
- [ ] Si se elige B, ¿hacemos temprano una síntesis de prueba de una etapa con rotador y FIFO en BRAM?
- [ ] ¿Confirmamos que el host calcula $\bar{H}$ con la f_s emulada (64, 72 o 120 GSa/s)?
- [ ] ¿Cómo llegan las muestras a la lógica: 8 complejas (I y Q) por ciclo, o 8 reales que después se convierten? ¿Con qué escala de pico?
- [ ] Al cambiar N, ¿cuántas muestras inválidas se aceptan y con qué valor? ¿El hardware informa k_0?
- [ ] ¿L_max = 4096 (N_max = 2048) es definitivo? Con N_max = 2048, 100 GBd a 2000 km no entra (R = 3923).
- [ ] ¿Qué presupuesto de NRMSE de cuantización aceptamos? Con W_d = 20 el error interno es de $2{,}5{\cdot}10^{-5}$ a $4{,}2{\cdot}10^{-5}$.
- [ ] ¿El vector de escalado queda configurable en ejecución o fijo por L en una tabla interna?
- [ ] ¿Alguien del equipo ya tiene una mariposa o un multiplicador complejo en RTL con modos de redondeo definidos para reutilizar?

Fuentes

- E. Börjeson y P. Larsson-Edefors, "Fiber-on-Chip: Digital Emulation of Channel Impairments for Real-Time DSP Evaluation", *J. Lightwave Technol.*, 41(3), 2023.
- J. Jiang, C. Xie y M. Chen, "Hardware implementation of configurable chromatic dispersion model for real-time optical fiber communication simulation systems", *J. Lightwave Technol.*, 44(12), 2026.
- AMD, *Fast Fourier Transform LogiCORE IP Product Guide* (PG109).
- AMD, *Zynq UltraScale+ RFSoC Data Sheet: Overview* (DS889).
- AMD Zynq UltraScale+ RFSoC ZCU208 Evaluation Kit (Mouser): ZU48DR, 8 ADC y 8 DAC de 14 bits.
- M. Garrido, "A Survey on Pipelined FFT Hardware Architectures", *J. Signal Processing Systems*, 94, 2022.
- M. Garrido, J. Grajal et al., "Pipelined Radix-2^k Feedforward FFT Architectures", *IEEE Trans. VLSI Syst.*
- S. He y M. Torkelson, "A new approach to pipeline FFT processor", *Proc. IPPS*, 1996.
- PDF del proyecto, *Fundamentos_CD_ISI_golden_model*, §4.7 y §4.8.